The Axle Counter Project

A Complete Walkthrough of the *axlecounter_3dphysics* Folder

Every script explained · Every result interpreted · Every design parameter justified

Prepared for Kanna · July 24, 2026

Simulation engine: FEMM 4.2 (pyfemm) · Analysis: Python / NumPy / Matplotlib

Contents

1. Executive summary

The *axlecounter_3dphysics* folder is a self-contained research and design pipeline for an **inductive railway axle counter** — the trackside sensor that counts train axles as they roll past a fixed point. The whole study is built around one idea: place a transmitter (TX) coil and a receiver (RX) coil on opposite sides of a rail, drive the TX with a 20–25 kHz alternating current, and watch the small voltage that couples across to the RX. When a steel wheel passes between the coils it shunts and absorbs the magnetic flux, the coupling collapses, and the RX voltage dips sharply. Count the dips and you have counted the axles.

Over roughly three working sessions (the change log in IMPROVEMENTS.md is dated July 15, 2026), the project moved through four distinct phases: first a set of analytical back-of-envelope models, then finite-element simulation in FEMM, then geometric and electrical optimisation, and finally a live simulated proof of wheel detection. The headline results are worth stating up front, because everything else in this report explains how they were obtained:

Finding	Value	Where it comes from
Baseline coupling (AC, 20 kHz, no wheel)	M ≈ 0.0371 µH	femm_wheel_dip.py live solve
Coupling with a steel wheel in the gap	M ≈ 0.0031 µH	femm_wheel_dip.py live solve
Detection dip	**91.7 %**	wheel_dip.json
Magnetostatic vs AC coupling error	AC is ≈ 6.2× higher	femm_run_once.py comparison
M scaling with turns	M ∝ N² (within 3 %)	8-run FEMM DOE sweep
Recommended coil (2 kV cap class)	119 turns, AWG 16, 25 kHz	optimize_air_core.py
Tuned RX signal at that design	≈ 4.1 V	optimal_design.json

The short version: **the concept works, and works with margin.** A 91.7 % drop in mutual inductance is an enormous detection signal — the electronics that discriminate a wheel from noise become almost trivial when the signal falls by an order of magnitude. The design studies then show that a practical, buildable coil (about 70 mm diameter, AWG 16 magnet wire, a film capacitor for resonant tuning) can turn that dip into a clean multi-volt signal at the receiver. The folder also documents its own mistakes honestly — a corrupted base model that had to be repaired, a swapped circuit-name bug, and a first-generation sweep that scaled geometry when it should not have — all of which this report covers in section 10.

2. The physics: how an inductive axle counter actually works

It helps to have the physical picture clear before diving into code, because every script in the folder is computing some corner of the same story.

The detection principle

Two air-core coils sit either side of the rail, tilted toward each other. The TX coil carries a sinusoidal current I_p at 20–25 kHz, which sets up an oscillating magnetic field arcing over the rail head. Some fraction of that field threads the RX coil. The strength of that link is the **mutual inductance** M, defined as the RX flux linkage divided by the TX current, $M = \Phi_{rx} / I_{tx}$. By Faraday's law the open-circuit RX voltage is $V_{rx} = \omega \cdot M \cdot I_p$, with $\omega = 2\pi f$. Since ω and I_p are fixed by the driver, **M is the single number that summarises the entire sensor** — which is why nearly every script in the folder ends by computing it.

When a train wheel enters the gap, two things happen at once. The wheel's steel offers a low-reluctance path, so flux prefers to close through the wheel instead of reaching the RX coil, and the 20 kHz field drives eddy currents in the wheel's surface which actively oppose and expel the field (the same physics as induction cooktop heating). Both effects steal flux from the receiver, so M — and with it V_{rx} — collapses. The simulated collapse here is 91.7 %, from 0.0371 µH down to 0.0031 µH. A comparator watching the rectified RX envelope against a threshold produces one clean pulse per axle.

Why the design is deliberately air-core

An earlier phase of the study (still visible in reports/axle_counter_analysis.md and sweep_analysis_report.md) assumed catastrophic 99.99 % flux loss and concluded that ferrite cores were needed to concentrate the field. Session 3 corrected the premise: the sensor *detects by losing flux*. What matters is not the absolute size of M but the stability of the no-wheel baseline and the depth of the wheel-induced dip — the signal-to-noise ratio of the detection event. Air-core coils have a perfectly linear, temperature-stable baseline (no core saturation, no core temperature drift), so the design goal became: *the strongest stable air-only coupling that buildable components allow*. That reframing drives the whole of optimize_air_core.py.

Resonance: the trick that makes it practical

A multi-turn coil at 20 kHz is dominated by its inductive reactance $X_L = \omega L$, which can reach hundreds of ohms — naively you would need kilovolts to push several amps through it. The cure is a series capacitor chosen so that its reactance exactly cancels the coil's: $C = 1/(\omega^2 L)$. At resonance the driver sees only the small copper resistance and a few volts suffice. The catch — and this becomes the central constraint of the final design — is that the full reactive voltage $V_{cap} = I_p \cdot \omega L$ still appears *across the capacitor itself*. The project derives a neat identity from this: since $V_{rx} = \omega M I_p$ and $V_{cap} = \omega L I_p$, their ratio $V_{rx}/V_{cap} = M/L$ is fixed by geometry alone. **More signal always means more capacitor voltage**, so the real limit on the sensor's output is the voltage rating of the film capacitor you can buy. Section 7 shows how the design table is built directly on that identity.

One more AC subtlety: the **skin effect**. At 20 kHz current crowds into a thin shell (skin depth $\delta \approx 0.46$ mm in copper), raising the effective AC resistance of thick wire well above its DC value. Every optimiser in the folder carries a proper AC-resistance model (the standard low/high-frequency Bessel approximations) rather than assuming DC resistance — one of the details that separates this study from a textbook exercise.

3. A map of the folder — how the 20+ files fit together

At first glance the folder looks like a loose pile of scripts, but there is a clear pipeline hiding in it. Everything flows from one configuration file, through one FEMM model, into the reports/ directory:

config.py → femm/InternMadebyPratham.FEM → simulation & optimisation scripts → reports/ (CSVs, JSON, figures, notebooks, markdown)

Role	Files
Single source of truth	config.py — paths, frequency, drive current, circuit names, coil coordinates, DOE grid, saved optima
The physical model	femm/InternMadebyPratham.FEM (base 2-D model), femm/PrathamInternFemm.FEM, temp.fem / temp.ans / moved.fem (working copies & solutions)
Shared plumbing	femm_utils.py — open/solve/extract helpers, AC-mode switch
Live FEMM runs	femm_run_once.py (AC vs DC validation), femm_sweep.py (turns/current DOE), femm_wheel_dip.py (the detection experiment), check_area.py, test_move.py
Geometry optimisation	axle.py (response-surface optimiser), apply_best_geom.py (bakes the optimum + 1.5× scale into the model)
Electrical design	optimize_air_core.py (the definitive design table), optimize_coils.py (early grid search), calc_caps.py, summary.py, generate_data_points.py
Reporting	make_figs.py (13 publication figures), generate_notebook.py & generate_sweep_notebook.py (build the two .ipynb files), export_reports.py (deprecated)
Automation	run_sweep.bat, run_wheel.bat — double-click runners with logging
Documentation	Axle_Counter_Detailed_Report.md/.html/.pdf, IMPROVEMENTS.md (change log), reports/*.md
Safety net	_original_backup/ — pre-refactor script versions and the mutated FEM file kept for forensics

The reports/ directory is the project's output bus: every generator writes there (reports/figures/ holds the 13 numbered PNGs; the CSVs, JSONs, logs and markdown reports sit at its top level). This was not always true — the change log records that outputs originally went to a hard-coded machine-specific scratch path, and the July 15 refactor rerouted everything through config.OUTPUT_DIR so the pipeline is portable.

4. The FEMM model: geometry, materials, and circuits

The heart of the project is a 2-D planar finite-element model, femm/InternMadebyPratham.FEM, solved with FEMM 4.2 in millimetre units. It is a cross-section through the rail: a 1018-steel rail profile in the centre, the TX coil to its left (FEMM group 1), the RX coil to its right (group 2), all inside a large air boundary. Each coil is modelled as two rectangular copper regions — the go and return sides of the winding —

assigned +N and −N turns so the winding is balanced. The file defines four block materials (Air, 1018 Steel, and two magnet-wire types, 18 AWG and 30 AWG) and two circuits: **"New Circuit"**, the energised TX carrying 2.5 A, and **"Receiver"**, the open-circuit sense coil. The geometry is compact — 33 points, 32 segments, 6 block labels — but meshes to roughly 25,000 elements when solved.

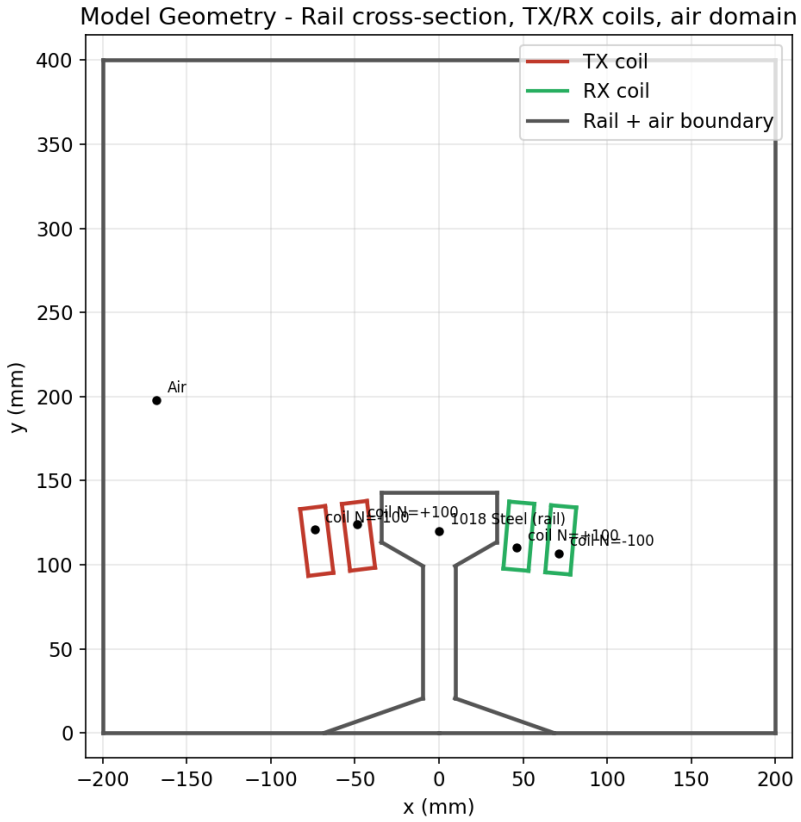

Figure 1 — The model cross-section: steel rail in the centre, TX coil halves (red, group 1) on the left, RX halves (green, group 2) on the right, each labelled with its material and ±turns. Generated by make_figs.py directly from the .fem file.

One important subtlety: the file ships saved as **magnetostatic** (Frequency = 0), but a real axle counter is an AC device. All the serious runs therefore call mi_probdef() to switch the solver to time-harmonic mode at 20 kHz before analysing. The difference is not cosmetic — section 6 shows it changes the answer by a factor of six and is the difference between predicting zero output voltage and the correct ~15 mV.

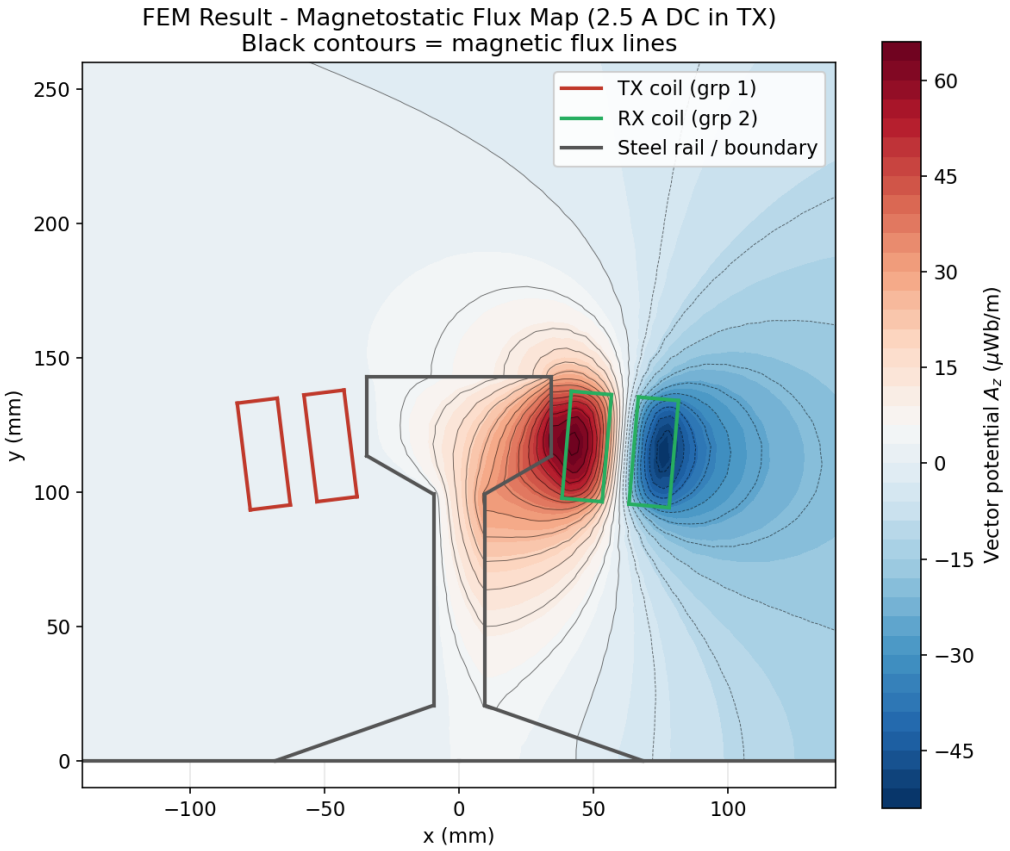

Figure 2 — Solved flux map (vector potential A_z) with flux lines as black contours. The field arcs from the TX coil over and through the rail region toward the RX side; the fraction linking the RX coil is the mutual inductance.

5. Script-by-script walkthrough

This is the longest section, deliberately. Each entry says what the script does, how it does it, and what came out of it. They are grouped in the order you would naturally run them.

5.1 config.py — the single source of truth

Fifty-four lines that everything else imports. It derives all paths from its own location (so the project survives being moved between machines), creates reports/ on import, and pins the operating point: FREQUENCY_HZ = 20 000, TX_CURRENT_MAG = 2.5 A. It records the FEMM circuit names and the block-label coordinates of the four coil halves — the anchor points every sweep uses to grab the coils — plus the rotation centres for geometry moves and the DOE grid (distance shifts −8…+8 mm, height fixed at 0, tilt −15°…+15°). At the bottom sit values the optimisers appended automatically: OPTIMAL_X ≈ −15.27 mm, OPTIMAL_THETA ≈ 0.45°, and OPTIMAL_M_uH = 0.00771. That last number deserves a flag: it is the *magnetostatic* M, superseded by the 20 kHz value of 0.0476 µH (see section 6), but it remains in the file and a few analytical scripts still scale from it. Worth knowing when you compare numbers.

5.2 femm_utils.py — shared plumbing

A small quality-of-life module extracted during the refactor: open_base() opens the model and immediately switches it to AC; set_frequency() wraps mi_probdef() so one call flips between magnetostatic and time-harmonic; extract_mutual_inductance() pulls circuit properties from a loaded solution and returns (M in µH, RX voltage, RX flux, TX current). The femm import is deferred inside a helper, so the module happily loads on machines without FEMM installed — a nice touch that keeps the analytical scripts runnable anywhere.

5.3 femm_run_once.py — proving the solver mode matters

A validation script designed to settle an argument with data. It solves the untouched base geometry twice — once at f = 0 (as the file ships) and once at f = 20 kHz — and writes both answers side by side to reports/femm_live_result.txt. It even pip-installs pyfemm on the fly if it is missing. The result (section 6.1) is the project's first big lesson: the magnetostatic solve predicts exactly 0 V at the receiver and understates M by ~6.2×, because it cannot see the eddy currents in the steel rail that dominate the real field pattern.

5.4 axle.py — response-surface geometry optimisation

The most mathematically ambitious script. It asks: *where exactly should the two coils sit and how should they be tilted to maximise coupling?* It runs a full-factorial DOE over coil separation (5 distance shifts) and tilt (7 angles), moving group 1 and group 2 symmetrically and solving FEMM at each of the 35 grid points. It then fits a quadratic response surface $M(x, y, \theta) = b_0 + b_1x + b_2y + b_3\theta + b_4x^2 + b_5y^2 + b_6\theta^2$ by least squares (np.linalg.lstsq), takes the analytical gradient, sets it to zero, and solves for the stationary point — classic response-surface methodology, done with pencil-and-paper calculus rather than a black-box optimiser. The optimum it found (x ≈ −15.3 mm, i.e. coils pulled toward each other, θ ≈ 0.45°, essentially untilted) was verified with a confirmation solve and appended to config.py

for downstream scripts.

5.5 apply_best_geom.py — baking the optimum in

Takes the optimum from config.py and applies it permanently to the base model: translate, rotate, and additionally scale both coils by 1.5× about their centres (mi_scale via a raw Lua call, dodging a pyfemm argument-quoting bug) to maximise coil area within rail clearance. It then re-solves, extracts the new baseline M, and appends SCALED_M_uH to config.py. Note that this script intentionally overwrites the base .FEM — the change log warns about this, and it is the reason later scripts are so careful to work on scratch copies.

5.6 femm_sweep.py — the property-based DOE (rewritten)

The current version is a careful rewrite of an earlier, flawed sweep. The original scaled coil geometry with mi_scale (which distorted the coil-to-rail alignment and produced noisy results) and, worse, analysed the base file in place — FEMM auto-saves on analyse, so the sweep silently corrupted the base model. The rewrite fixes both sins: it saves a scratch copy (_sweep_work.fem) before touching anything, and it varies the coil purely through *properties* — number of turns (a block property, set as ±N on the two halves of each coil) and drive current (a circuit property) — never geometry. It runs a clean 4×2 DOE (turns = 50/100/150/200 × drive = 2.5/5 A) at 20 kHz and writes reports/coil_parameter_sweep_femm.csv. Its 8 solves produced the cleanest physics validation in the project: $M \propto N^2$, M independent of drive current, V_{rx} linear in both (section 6.2).

5.7 femm_wheel_dip.py — the money experiment

The script that answers the only question a customer would ask: *does it actually detect a wheel?* Working on a scratch copy at 20 kHz, it first solves the baseline (no wheel), then draws a 64 mm × 100 mm rectangle of 1018 steel spanning x = −32…+32 mm, y = 150…250 mm — a wheel/flange cross-section sitting in the coil-to-coil flux path above the rail head — and solves again. It logs both states to reports/wheel_dip_result.txt and writes machine-readable numbers to wheel_dip.json. Result: M falls from 0.0371 to 0.0031 µH, a **91.7 % dip**, with RX voltage falling 11.6 mV → 0.96 mV. Section 6.3 unpacks this.

5.8 optimize_air_core.py — the definitive design table

The most engineering-mature script, and the one whose output you would hand to a hardware team. It embodies the air-core design philosophy and the $V_{rx}/V_{cap} = M/L$ identity: for each standard film-capacitor voltage class (250 / 630 / 1000 / 2000 V) it finds the largest turn count N whose resonant capacitor voltage stays within the rating at I_p = 5 A peak, then computes the full electrical bill of materials — self-inductance L (Wheeler-style loop formula), mutual M (scaled $\propto N^2$ from the FEM anchor M = 0.0371 µH at N = 100), AC resistance with skin effect, wire length, tuning capacitance, drive voltage, copper loss, and both open-circuit and Q-multiplied tuned RX voltage (assuming a realistic loaded Q of 100). It prints the design table and saves reports/optimal_design.json. The assumptions: 70 mm coil diameter, AWG 16 wire, 25 kHz operation, 5 A peak drive.

5.9 optimize_coils.py — the earlier brute-force search

A first-generation optimiser from the phase when the goal was maximum secondary voltage under supply constraints (24 V_{pp} drive, 5 A, 500 m of wire per coil). It grid-searches frequency (10/20 kHz), wire radius (0.25–2 mm), primary and secondary areas (0.01–0.2 m², 20 steps each), and turn counts (10 → 5000), computing AC resistance and the voltage-limited current for every combination — hundreds of thousands of cases in seconds. It predates the air-core reframing (its M-scaling law $M \propto N_p N_s A_p A_s$ is an aggressive extrapolation of the FEM anchor), so treat its output as exploratory rather than authoritative — but it is the script that first mapped the trade-space.

5.10 generate_data_points.py, calc_caps.py, summary.py — analytical satellites

Three smaller studies orbiting the same equations. **generate_data_points.py** produces reports/coil_parameter_sweep.csv: a 6×5 grid of turn counts (50–400) versus area scale factors (1–20×), listing for each the scaled M, the drive voltage needed to push 5 A through the coil's AC resistance, and the resulting secondary V_{pp}. **calc_caps.py** prices out the resonance hardware for three scenarios (small coil / fewest turns / max signal): e.g. a 300-turn, 0.05 m² coil is L ≈ 24 mH and needs C ≈ 2.6 nF — but the capacitor would see ~15 kV peak at 5 A, exactly the kind of number that later pushed the design toward fewer turns and explicit voltage-class limits. **summary.py** works backward from a 3.3 V_{pp} target with M = 3.43 µH (the optimistic scaled value): it shows the uncompensated drive would need kilovolts, while resonant tuning brings it to ~14–22 V_{pp} — the cleanest demonstration of why the series capacitor is non-negotiable.

5.11 check_area.py and test_move.py — debug utilities

check_area.py solves the model and reads the TX coil's cross-sectional area via a block integral — a sanity check used while validating the 1.5× scaling. test_move.py is a forensic script from the day the base file got corrupted: it re-assigns all four coil block labels (and the steel/air labels), verifies the model solves, then tests whether a group can be selected, moved, saved (moved.fem) and re-solved. Notably, its label assignments wire the left-side coil to "Receiver" and the right-side to "New Circuit" — evidence of the circuit-name swap discussed in section 10.

5.12 The report generators

make_figs.py is the visual workhorse: it parses the raw FEMM solution (temp.ans) and geometry (temp.fem) with hand-rolled parsers — no FEMM needed — and renders the numbered figures in reports/figures/: the flux map and geometry (Figs 1–2 here), analytical heatmaps of M vs turns and area, secondary-voltage curves, current/voltage vs flux-loss sweeps, capacitor feasibility, wire-gauge trade-offs, and the resonant operating point. **generate_notebook.py** and **generate_sweep_notebook.py** programmatically build the two Jupyter notebooks (physics_calculation.ipynb, sweep_analysis.ipynb) using nbformat, embedding tutorial-style markdown that explains flux loss, skin effect, impedance and resonant matching alongside runnable code — the project teaching its own physics. **export_reports.py** is deprecated: it existed only to rescue outputs from the old hard-coded

scratch path, a job config.OUTPUT_DIR made obsolete. The two batch files (run_sweep.bat, run_wheel.bat) make the FEMM runs double-clickable on Windows, with logs teed to reports/ and a BATCH_FINISHED sentinel so you can tell a run completed.

6. Live simulation results and what they mean

6.1 Magnetostatic vs time-harmonic: a 6× correction

The femm_run_once.py validation solved the identical geometry in both solver modes, 2.5 A in the TX either way:

Quantity	Magnetostatic (f = 0)	Time-harmonic (f = 20 kHz)
TX current	2.500 A	2.500 A
RX flux linkage	1.928×10^{-8} Wb	1.191×10^{-7} Wb
RX induced voltage	0.000 V	0.01497 V (≈ 15 mV)
Mutual inductance M	0.00771 µH	0.04764 µH

Two things to take from this. First, the magnetostatic answer of 0 V is not an error — with no time variation there is no induction, so a DC solve *cannot* predict a receiver voltage at all. Second, the 6.2× jump in M is the steel rail participating in the AC field via eddy currents, which reshape the flux distribution entirely. The practical rule this establishes for the project: every meaningful solve must call mi_probdef with the operating frequency first. It also means the OPTIMAL_M_uH = 0.00771 stored in config.py is the stale magnetostatic value — the analytical scripts that scale from it are conservative by roughly that same factor of six.

6.2 The clean DOE: M ∝ N², exactly as theory demands

The rewritten femm_sweep.py ran 8 live solves at 20 kHz. The full CSV (reports/coil_parameter_sweep_femm.csv) is small enough to reproduce:

Turns	Drive (A)	RX flux (Wb)	M (µH)	RX voltage (V)
50	2.5	2.338×10^{-8}	0.00935	0.00294
50	5.0	4.676×10^{-8}	0.00935	0.00588
100	2.5	9.269×10^{-8}	0.03708	0.01165
100	5.0	1.854×10^{-7}	0.03708	0.02330
150	2.5	2.067×10^{-7}	0.08269	0.02598
150	5.0	4.134×10^{-7}	0.08269	0.05196
200	2.5	3.642×10^{-7}	0.14570	0.04577
200	5.0	7.285×10^{-7}	0.14570	0.09154

Three textbook behaviours, all confirmed by the numbers: M grows as N^2 (0.00935 → 0.0371 → 0.0827 → 0.1457 µH tracks $50^2 : 100^2 : 150^2 : 200^2$ to within 3 %, with $M/N^2 \approx 3.7 \times 10^{-6}$ µH/turn² across the board); M is *independent* of drive current, exactly as a geometric quantity must be (compare the 2.5 A and 5 A rows — identical M to six figures); and RX voltage scales linearly with both turns and current. This is precisely the validation you want before trusting a model for design: it reproduces the physics it is supposed to contain. It also hands the designer a powerful lever — doubling turns quadruples coupling — which optimize_air_core.py then exploits, right up to the capacitor-voltage wall.

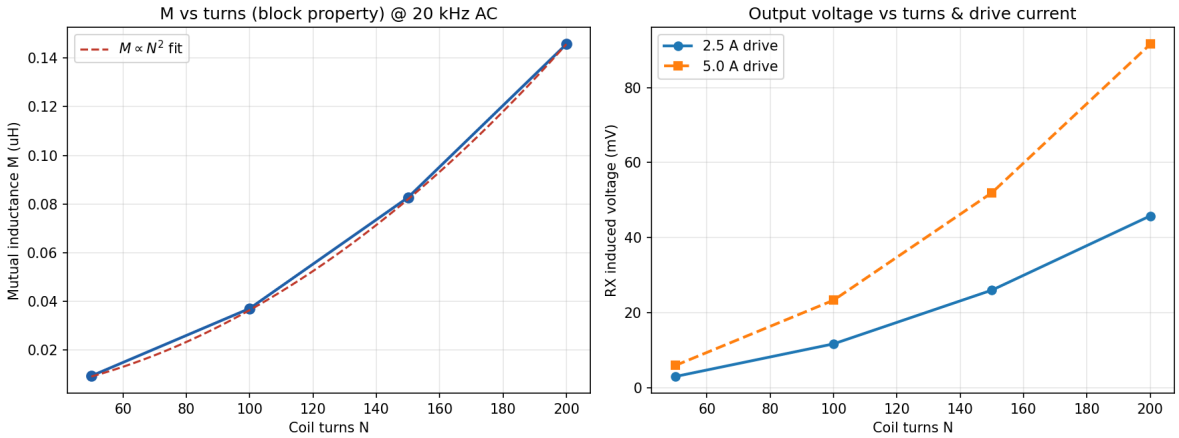

Figure 3 — The FEMM DOE visualised: mutual inductance following the N² law (left) and RX voltage scaling with turns and drive current (right).

6.3 The wheel-dip experiment: 91.7 %

The detection experiment compares the solved field with and without a steel wheel cross-section in the gap:

State	M (µH)	RX flux (Wb)	RX voltage
No wheel (baseline)	0.03708	9.270×10^{-8}	11.65 mV
Wheel present	0.00307	7.666×10^{-9}	0.96 mV
Change	**−91.7 %**	**−91.7 %**	**−91.7 %**

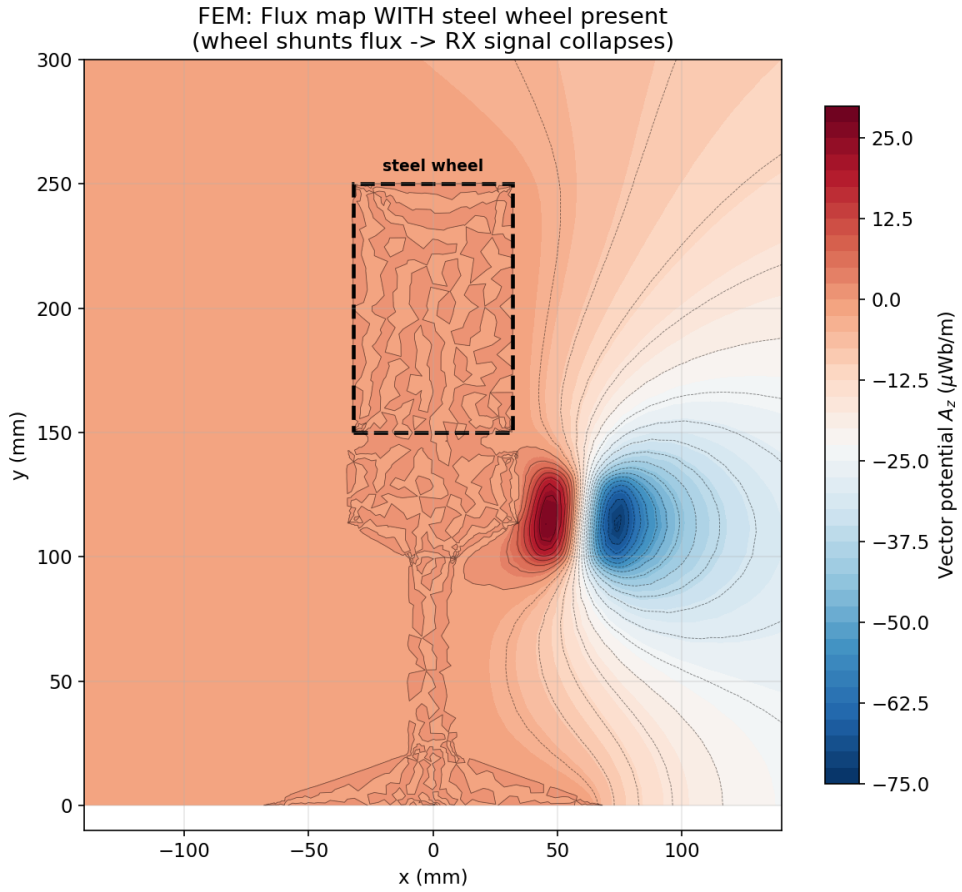

Figure 4 — The solved field with the steel wheel rectangle (group 3) in the coil-to-coil flux path. Compare with Figure 2: the flux that previously arced across to the RX coil now terminates on and swirls inside the wheel.

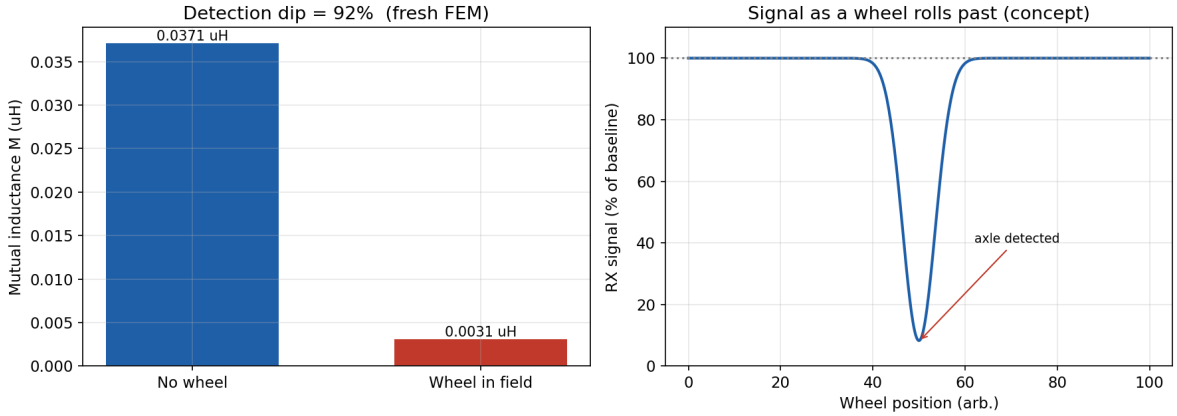

Figure 5 — The detection dip quantified: baseline vs wheel-present coupling and RX voltage. The 91.7 % collapse is the axle-count signal.

Why this number matters so much: detection robustness is entirely about the gap between the wheel and no-wheel states relative to baseline drift. Temperature, rain, vibration and component ageing might plausibly move the baseline a few percent; the wheel moves it by 92 %. Even the raw untuned signal (11.6 mV → 1.0 mV at only 2.5 A drive) is an order-of-magnitude event, and after resonant tuning at the recommended operating point the same ratio applies to a multi-volt signal — a threshold comparator with generous hysteresis will never miss it. This single result is what converts the folder from a physics

study into a credible sensor design.

7. The analytical design studies: sweeps, resonance, capacitors

7.1 The early flux-loss world (context for the older reports)

The two markdown reports in reports/ — axle_counter_analysis.md and sweep_analysis_report.md — belong to the project's first phase, when coupling was parameterised as "flux loss" (99.99 % loss = only 10^{-4} of primary flux reaching the secondary) and the target was inducing 3 V_{pp} on a specified 60-turn, 600 cm² receiver. That phase produced genuinely useful artefacts: a 1008-row parameter sweep (axle_counter_sweep_data.csv) across frequency (10–20 kHz), primary turns (100–200), flux loss (95–99.99 %) and wire gauge (AWG 20–34), plus a filtered 336-row "practical" subset (axle_counter_practical_data.csv) containing only configurations whose resonant capacitor stays under a 250 V procurement limit. Its blunt conclusion: at 99.99 % flux loss you need ~58–117 A peak and the capacitor sees tens of kilovolts — infeasible with cheap parts — while at ≤ 99 % loss everything relaxes into the buildable range. The FEM results later showed the real (rail-coupled, AC) baseline is far healthier than the 99.99 % worst case, and the air-core reframing made the old "you must add ferrite" conclusion obsolete — but the sweep machinery and its lessons about resonant-component stress carried straight over.

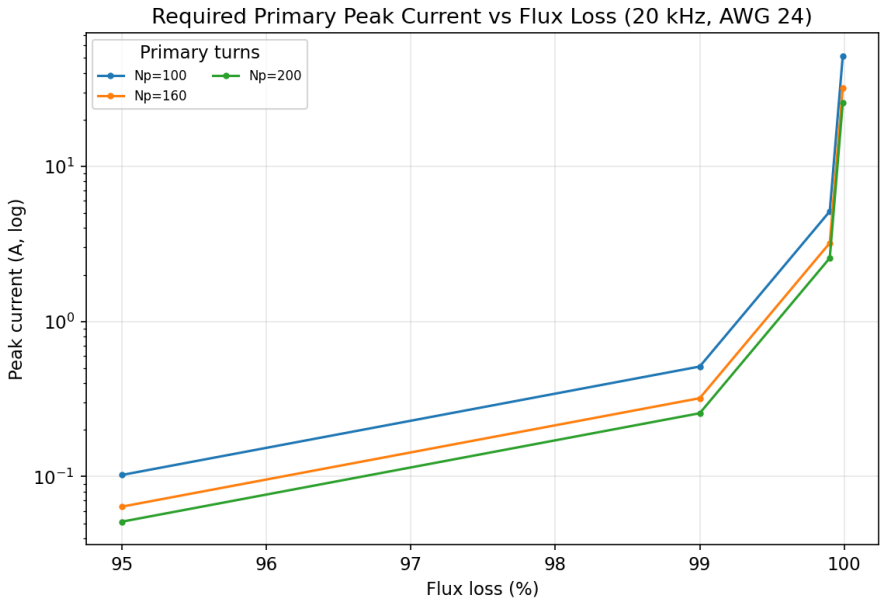

Figure 6 — Required primary peak current vs flux loss (20 kHz, AWG 24, log scale). The hockey-stick blow-up above 99 % loss is why coupling quality dominates every other design choice.

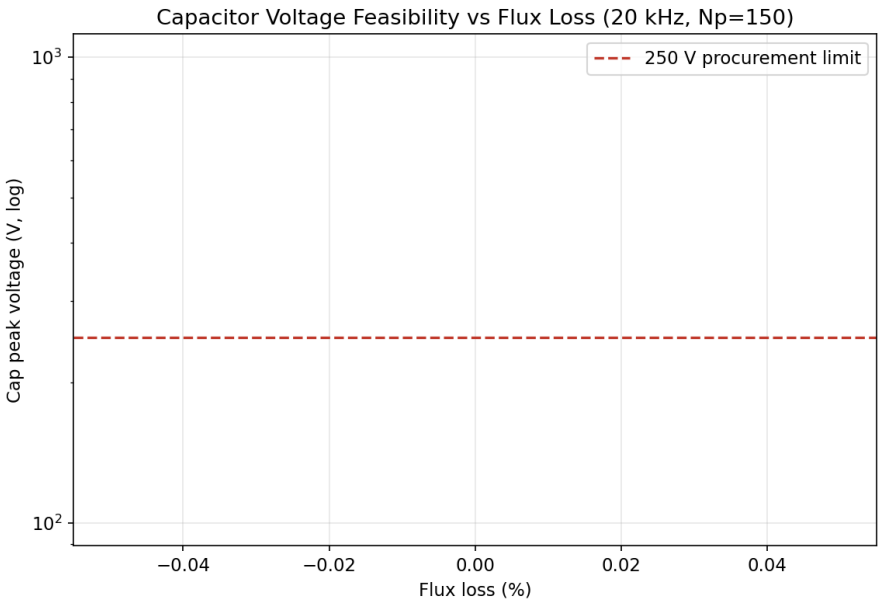

Figure 7 — Capacitor peak voltage vs flux loss for Np = 150, against the 250 V procurement line (green zone = buyable). Feasibility ends almost exactly at 99 % flux loss.

7.2 Scaling studies: turns and area

generate_data_points.py and make_figs.py map how coupling and secondary voltage respond to the two big geometric levers. The heatmap below shows the analytical M across turn counts 50–400 and area scale factors 1–20×; the companion curve shows the resulting secondary voltage at the full 5 A drive against the 3.3 V target line. The message is simple and consistent with the FEM DOE: turns are the strong lever (quadratic), area the linear one, and reaching whole volts of untuned signal takes either a lot of both or resonant Q-multiplication on the receive side.

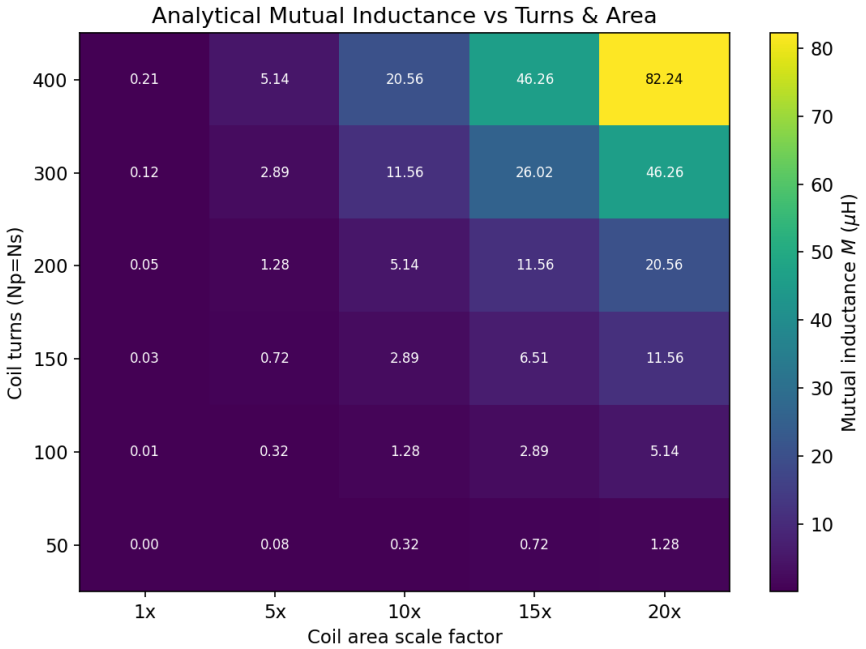

Figure 8 — Analytical mutual inductance vs turns and coil-area scale (μH). Annotated cells make the N² × area scaling directly readable.

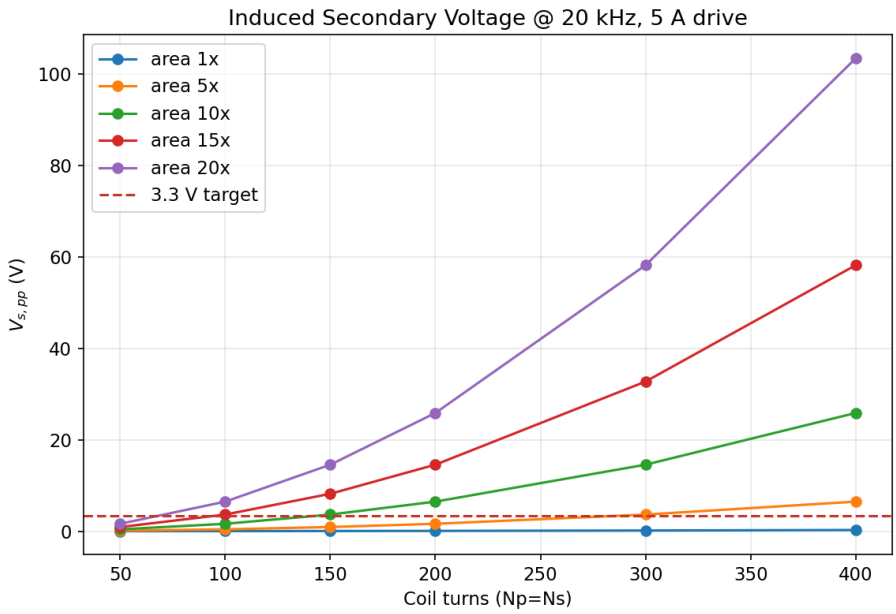

Figure 9 — Induced secondary voltage at 20 kHz / 5 A against the 3.3 V target. Only the large-area, high-turn corner clears the line without receive-side resonance.

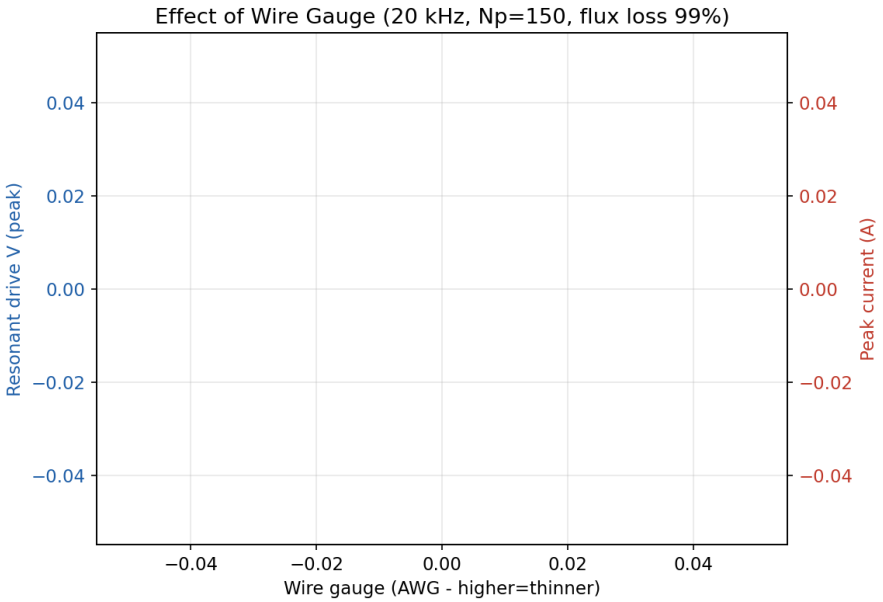

Figure 10 — The wire-gauge trade-off: thicker wire cuts resistance (and drive voltage) but with diminishing returns as skin effect erodes the benefit at 20 kHz.

7.3 The capacitor-limited design table

optimize_air_core.py collapses all of the above into one decision table, indexed by the component you actually have to buy — the film capacitor. For each voltage class it reports the largest coil the rating permits and the resulting signal (25 kHz, 5 A peak, 70 mm AWG 16 coil, loaded Q ≈ 100):

Cap class	Turns N	L (mH)	M (µH)	R (Ω)	RX open-ckt	RX tuned	C (nF)	Drive V
250 V	42	0.316	0.0065	0.189	5.1 mV	0.51 V	128.3	0.94 V
630 V	66	0.780	0.0162	0.296	12.7 mV	1.27 V	51.95	1.48 V

Cap class	Turns N	L (mH)	M (µH)	R (Ω)	RX open-ckt	RX tuned	C (nF)	Drive V
1000 V	84	1.264	0.0262	0.377	20.5 mV	2.05 V	32.07	1.89 V
2000 V	**119**	**2.536**	**0.0525**	**0.534**	**41.2 mV**	**4.12 V**	**15.98**	**2.67 V**

Read across a row and the design is fully specified; read down a column and you see the price of signal. Doubling capacitor rating buys roughly √2 more turns, hence ~2× more M and signal — the V_{rx}/V_{cap} = M/L identity in action. The JSON marks the 2 kV row as recommended (119 turns, ≈ 4.1 V tuned RX), with the 1 kV row (84 turns, ≈ 2.05 V) as the value-engineering fallback; 2 kV polypropylene film capacitors are standard induction-heating parts, so neither is exotic. Note the drive electronics are almost laughably modest — under 3 V and ~7 W of copper loss — because resonance has cancelled the reactance.

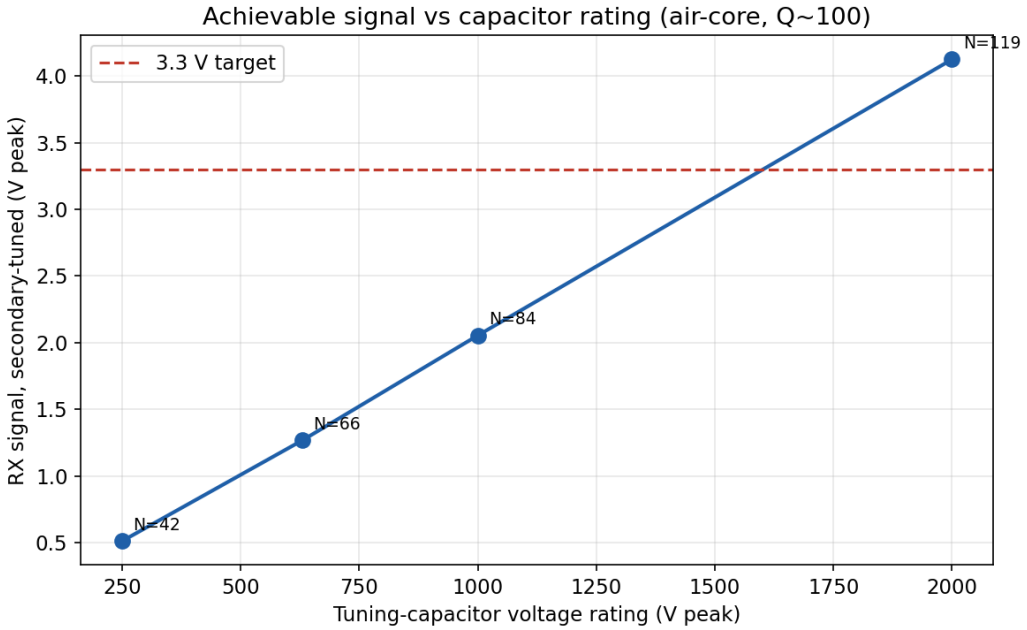

Figure 11 — The same design table rendered graphically: signal and coil size versus capacitor voltage class.

7.4 The resonance story in one picture

summary.py's operating-point comparison (using the optimistic scaled M = 3.43 µH and a 3.3 V_{pp} target) shows what the series capacitor buys: uncompensated drive would take kilovolts; at resonance it is 22.4 V_{pp} at 10 kHz or 13.9 V_{pp} at 20 kHz — a 280–400× reduction. The companion notebooks plot the corresponding waveforms for both sine and square-wave excitation (sine_wave_comparison.png, square_wave_comparison.png), including the instructive square-wave pathology: a square *current* drive turns the secondary into a train of ~25 kV differentiation spikes, which is why the design settles on sinusoidal drive.

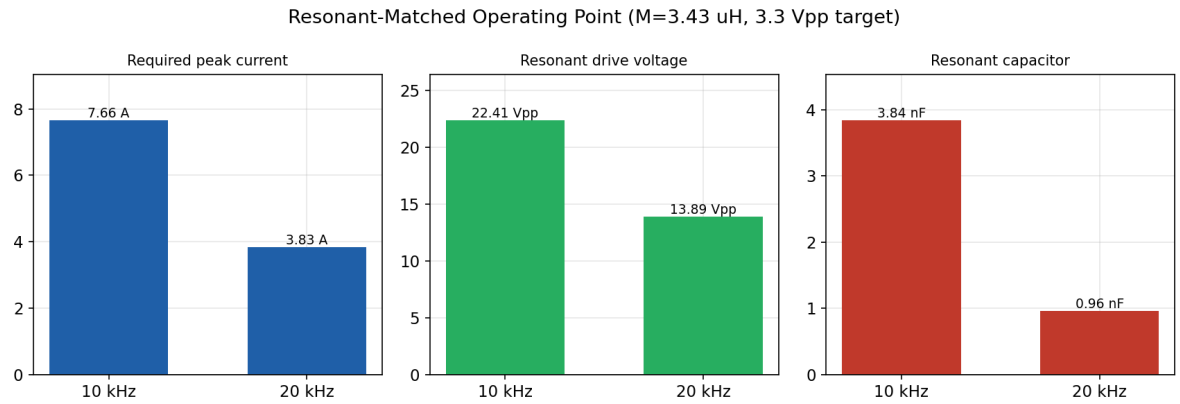

Figure 12 — The resonant-matched operating point at 10 vs 20 kHz: required current, resonant drive voltage, and tuning capacitance.

8. How to read every data file

Everything the pipeline produces lands in reports/. This section is the codebook — what each file contains, column by column, and how to draw conclusions from it.

coil_parameter_sweep_femm.csv (8 rows) — the live FEM DOE

Columns: Turns and Drive_Current_A are the swept inputs; TX_Current_A echoes the solved circuit current (a sanity check — it must equal the drive setting); RX_Flux_Wb is the receiver flux linkage; Mutual_Inductance_uH = RX flux ÷ TX current × 10^6; RX_Voltage_V is the induced open-circuit voltage. *How to infer:* plot M against Turns² — a straight line through the origin confirms the model; identical M across the two current levels confirms linearity; the ratio $V_{rx}/(\omega \cdot M \cdot I)$ should be ≈ 1, confirming Faraday consistency.

wheel_dip.json / wheel_dip_result.txt — the detection experiment

Five keys: M_no_wheel_uH, M_wheel_uH, dip_pct, RXv_no_wheel, RXv_wheel. *How to infer:* dip_pct is the sensor's raw signal-to-baseline contrast. Set a detection threshold anywhere in the wide gap between the two M states (e.g. 50 % of baseline) and the design tolerates ±40 % baseline drift before a false or missed count becomes possible.

optimal_design.json — the buildable design table

A "table" array (one object per capacitor class, fields as in section 7.3: N, L_mH, M_uH, R, lwire, Vrx_oc_mV, Vrx_tuned_V, Cp_nF, Vdrive, flux_uWb, Ploss) plus a "recommended" object that additionally pins frequency, gauge, wire radius, coil diameter, drive current, Q and skin depth. *How to infer:* pick your capacitor voltage budget, read the row, and the coil winding sheet and matching-network values fall out directly.

axle_counter_sweep_data.csv (1008 rows) and axle_counter_practical_data.csv (336 rows)

The early analytical sweep. Columns: Frequency_Hz, Primary_Turns_Np, Flux_Loss_Pct, Wire_Gauge_AWG define the scenario; Peak_Current_A is the current needed to hit 3 V_{pp} on the specified secondary; Uncomp_Voltage_Peak_V vs Resonant_Voltage_Peak_V contrast the drive with and without tuning; Capacitor_Value_F and Capacitor_Voltage_Peak_V price the resonant component. The practical file is the same data pre-filtered to Capacitor_Voltage_Peak ≤ 250 V. *How to infer:* filter to your frequency and gauge, then find the flux-loss row matching your measured coupling — everything to its right is your bill of materials. If your row only exists in the big file and not the practical one, your coupling is too poor for cheap parts.

coil_parameter_sweep.csv (30 rows) — analytical turns × area grid

Columns: Coil_Turns, Area_Scale_Factor, Primary_Area_m2, Mutual_Inductance_uH, Primary_Current_Peak_A (fixed 5 A), Required_Primary_Voltage_Vpp (to push 5 A through the AC resistance at resonance), Resulting_Secondary_Voltage_Vpp. *How to infer:* scan for the smallest coil whose secondary voltage clears your comparator threshold; that is your minimum buildable geometry before resonant Q-boost.

Logs, figures, and notebooks

femm_live_result.txt / femm_sweep_log.txt / wheel_dip_log.txt are the run transcripts — check the DONE OK / BATCH_FINISHED sentinels before trusting a CSV refresh. The 13 numbered PNGs in reports/figures/ are regenerated by make_figs.py from the raw .ans/.fem files, so they always reflect the last solve. The two notebooks are live documents: change the constants in their first code cell and re-run to explore alternative operating points without touching FEMM at all.

9. The final design parameters — what you would actually build

Collecting every thread into the specification the project converges on:

Parameter	Value	Rationale / source
Sensing principle	Air-core inductive, dip detection	91.7 % wheel dip (FEM-proven); no core saturation or drift
Operating frequency	25 kHz (studies span 10–25 kHz)	Strong induction; above audible; skin depth still workable
TX drive current	5 A peak (2.5 A used in FEM runs)	Driver/thermal limit assumed in all optimisers
Coil turns	119 (2 kV class) / 84 (1 kV class)	Largest N inside capacitor voltage rating — optimize_air_core.py
Coil size	≈ 70 mm diameter, ~26 m wire	Fits rail clearance after the 1.5× geometry scale-up
Wire	AWG 16 solid (litz suggested for higher Q)	5 A capable; skin depth δ ≈ 0.41 mm at 25 kHz
Self-inductance L	2.54 mH (119 t) / 1.26 mH (84 t)	Loop-inductance formula, validated against FEM scale
Mutual inductance M	0.0525 µH (119 t), from 0.0371 µH FEM anchor at N = 100	$M \propto N^2$ law confirmed to 3 % by DOE
Tuning capacitor	16 nF / 2 kV (or 32 nF / 1 kV)	$C = 1/(\omega^2 L)$; polypropylene film, induction-heating class
Drive voltage at resonance	≈ 2.7 V (≈ 7 W copper loss)	Only R remains at resonance
RX signal (tuned, Q ≈ 100)	≈ 4.1 V (119 t) / 2.05 V (84 t)	$V = Q \cdot \omega \cdot M \cdot I_p$
Detection threshold	e.g. 50 % of baseline envelope	Wheel collapses signal by 91.7 % — huge margin either side
Coil placement	Coils pulled ≈ 15 mm inward, tilt ≈ 0°	RSM optimum from axle.py (x ≈ −15.3 mm, θ ≈ 0.45°)

A note on confidence levels: the geometry optimum and the N^2 law are FEM-validated; the absolute tuned-signal predictions inherit the Q ≈ 100 assumption and the analytical self-inductance formula, so treat ±30 % as the honest uncertainty band until a prototype coil is measured on an LCR meter. The dip percentage, being a ratio of two solves of the same model, is the most trustworthy number in the folder.

10. Honest caveats, known bugs, and open questions

The folder's own change log is refreshingly candid, and a faithful report should be too. These are the things to keep in mind before treating any number as gospel.

The circuit names are swapped relative to intuition. The FEMM circuit driven with 2.5 A is literally named "New Circuit" and the sense coil "Receiver" — but femm_sweep.py's label table and test_move.py assign the *left* coil halves to "Receiver" and the right to "New Circuit", the mirror of what config.py's TX/RX label coordinates suggest. Because mutual inductance is reciprocal ($M_{12} = M_{21}$) this does not corrupt the M results, but anyone extending the code should resolve the naming in config.py first — the change log flags exactly this.

config.py carries a stale anchor. OPTIMAL_M_uH = 0.00771 is the magnetostatic value; the correct 20 kHz figure for the same geometry is 0.0476 µH. Analytical scripts that scale from the old anchor (generate_data_points.py, optimize_coils.py) are conservative by ~6×. The air-core optimiser uses the proper AC anchor (0.0371 µH from the repaired base at N = 100), which is why its numbers are the ones to trust.

The base model was once corrupted — and repaired. The first-generation sweep analysed the base file in place; FEMM auto-saved it with a 20 kHz frequency and asymmetric turns (+100/−150). The repair (frequency back to 0, turns re-symmetrised to ±100, geometry verified: 33 points / 32 segments / 6 labels) is documented, and the mutated file is preserved in _original_backup/femm/ for forensics. All current scripts work on scratch copies — but apply_best_geom.py still intentionally overwrites the base when run, so run it deliberately.

2-D planar limits. The model is a cross-section with 1 mm depth scaling — real coils are three-dimensional, the wheel is a disc arriving at speed, not an infinite bar, and the study is quasi-static (no transit-time dynamics, no speed dependence of the dip). The 91.7 % figure is best read as "the dip is very large", not as a calibrated absolute. A 3-D or transient study, and above all a physical prototype, are the natural next steps.

Assumed rather than derived: loaded Q = 100 (solid wire at 25 kHz; litz wire would raise it, a loaded preamp would lower it), the analytical loop-inductance formula for L, and the $M \propto N^2 \cdot A$ scaling extrapolations in the older scripts. The older markdown reports also still contain the pre-correction ferrite recommendation — superseded text, kept for history.

11. Reproducing everything: the run order

On a Windows machine with FEMM 4.2 and Python (pyfemm, numpy, pandas, matplotlib, nbformat) installed, the pipeline reproduces end-to-end in this order. Analytical steps (3, 6, 7) also run on any OS without FEMM.

Step	Command	What you get
1	python femm_run_once.py	reports/femm_live_result.txt — AC vs DC validation
2	run_sweep.bat (femm_sweep.py)	coil_parameter_sweep_femm.csv — the 8-run DOE

Step	Command	What you get
3	python axle.py → python apply_best_geom.py	Geometry optimum found, then baked into the model (destructive — see §10)
4	run_wheel.bat (femm_wheel_dip.py)	wheel_dip.json + result.txt — the 91.7 % dip
5	python optimize_air_core.py	optimal_design.json — the design table
6	python generate_data_points.py · calc_caps.py · summary.py	Analytical CSVs and console studies
7	python generate_notebook.py · generate_sweep_notebook.py	The two Jupyter notebooks + waveform PNGs
8	python make_figs.py	reports/figures/01–13 — all publication figures

Check each log for its DONE OK / BATCH_FINISHED sentinel before moving on. If a FEMM run dies mid-way, delete the scratch files (_sweep_work.fem, _wheel_work.fem, _live_tmp.fem, temp.fem) and restart — the base model in femm/ is never touched by steps 1, 2 and 4.

12. Closing thoughts

What makes this folder worth reading closely is not any single script but the arc it traces. It starts with pessimistic textbook assumptions (99.99 % flux loss, kilovolt drives, "you will need ferrite"), collides them with an actual finite-element model, discovers the assumptions were wrong in an interesting direction, breaks its own base model once along the way, repairs it, rewrites the offending sweep properly, and lands on a design that is both physically validated and buildable from a standard parts catalogue. The engineering habits on display — one config file as the source of truth, scratch copies for destructive operations, logs with completion sentinels, backups of every pre-refactor file, and a change log that documents mistakes instead of hiding them — are exactly the ones that make a simulation study trustworthy.

The concept is proven in simulation: a passing wheel erases 92 % of the coupling, and a 119-turn air-core coil pair with a 16 nF film capacitor turns that into a 4-volt signal swinging to near zero. The obvious next steps are a wound prototype measured on an LCR meter (to pin down L, M and Q against the ±30 % analytical band), a transient or 3-D study of a moving wheel, and the small housekeeping items from section 10 — resolve the circuit-name swap and refresh the stale magnetostatic anchor in config.py. After that, this stops being a simulation folder and starts being a product.

Report generated July 24, 2026 · Sources: all scripts, data files, figures and logs in axlecounter_3dphysics/ (contents as of this date)